Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.18 / COMPANION WORKBOOK

Stand up the business services

Files, database, licensing & web — by documented paths

Give the users what the network exists for: SBD-FS01's role-based shares over verified SMB security, SBD-WEB01 serving internal HTTPS from a hardened, domain-joined Linux Apache, SBD-APPS01's licensing service and application API, and SBD-SQL01 running as a managed identity behind exactly one caller — every hop a documented, proven register row.

MATERIALS EDITION 1.4 / COURSE 1.18

Updated 14 October 2026 · sbd-business-services

NAME _____ / DATE _____

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the service paths and identities

02 Build SBD-FS01 with least-privilege shares

03 Build SBD-SQL01 behind its applications

04 Build SBD-APPS01 and SBD-WEB01

05 Reconcile, scan, and hand off

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

The users finally get what the network exists for: files, licensing, and an internal web application with its application tier and database. The model's rule governs the whole course — permit only documented business service paths — so every service arrives with its register rows, its service identity, and its certificate, and nothing reaches it any other way.

Prerequisites

- ☐ Complete courses 1.1–1.15: the domain with its DNS, time and hardened OUs, the course 1.6 PKI and its autoenrollment GPO, DHCP and the admin seats, SBD-REPO01 and its Linux client package, the logging pipeline with course 1.11's KDS root key, SBD-SPLUNK01 as the Linux server pattern, the fleet, the update rings, and the SBD-MGMT01 inventory.
- ☐ Provision the VLAN 40 hosts as virtual machines on SBD-HV05: SBD-FS01 (10.0.40.10), SBD-SQL01 (10.0.40.20) and SBD-APPS01 (10.0.40.30) on Windows Server, and SBD-WEB01 (10.0.40.40) on the Red Hat Enterprise Linux–family release course 1.9 chose, with Apache, to be joined to sbd.test with SSSD as SBD-SPLUNK01 was in course 1.12.
- ☐ Have the service-path register, the course 1.6 SBD Server Authentication template, course 1.9's client package and Linux assessment set, and course 1.4's SCC media within reach — every service here gets a documented path, a real certificate and a first baseline.

What this course builds

- Serve files through role-group permissions with verified SMB signing and encryption.
- Run the database as a managed identity, certificate-encrypted, with SBD-APPS01 as its only caller.
- Prove the fleet → web → application → database chain with every hop registered and every shortcut denied.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-FS01 — name, address, OS build	
SBD-SQL01 — name, address, OS build	
SBD-APPS01 — name, address, OS build	
SBD-WEB01 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 30 MIN

Design the service paths and identities

> **WHY THIS MATTERS** Four services, and before any of them exist: who talks to what over which port, which identity each service runs as, and which groups own access — the register and the directory answer all three before a role installs.

In this lesson you will

- Write the register rows for every business service path.
- Choose managed service identities instead of password accounts.
- Define the access groups that will own each service's data.

Follow the build

1. Write the service-path rows into the register, with the IDs the System Hub gives them and an owner each: the fleet to SBD-FS01 on SMB 445 (F05), to SBD-APPS01 on its licensing port, recorded per application (F12), and to SBD-WEB01 on HTTPS 443 (F13); SBD-WEB01 to SBD-APPS01 for the application API on HTTPS 443 (F21); and SBD-APPS01 — and only SBD-APPS01 — to SBD-SQL01 on TCP 1433 with TLS (F22). Add the explicit deny from SBD-WEB01 to SBD-SQL01 (F14): the web tier has no database dependency. All four servers share VLAN 40, so the router never sees their traffic to each other — each server's host firewall enforces those rows. Everything else toward VLAN 40 stays denied, including endpoint-to-SQL directly.
2. Add the rows every new server needs: administration from SBD-ADMIN-SRV01 (A02), and for the three Windows servers the domain rows (F01–F03, F09), Ring 3 updates (F06), event forwarding (F17), certificate publication (F18) and enrollment to SBD-SUBCA01 on the row course 1.6 confirmed. SBD-WEB01's Linux rows already name it: packages from SBD-REPO01 only (F19, HTTP with GPG-signed packages, course 1.9), syslog over TLS to the collector, SBD-SYSLOG01 (F07, TCP 6514), and time from both domain controllers (F09) — the same paths SBD-SPLUNK01 has used since course 1.12.
3. Choose the service identities: group managed service accounts (gMSAs) for the SQL and application services, so passwords rotate themselves. They rely on the forest's one KDS root key, which course 1.11 created: confirm it with Get-KdsRootKey and never create another. Each account is created from AdminJumpBox01 once its host has joined, allowed to that one computer, as course 1.11 did (lesson 3). Note delegated managed service accounts, introduced in the newest Windows Server release, as the migration path for any legacy service account this lab later inherits.
4. Create the access groups under the course 1.5 Groups OU from AdminJumpBox01 — per-share and per-application role groups — because permissions will be granted to roles, never to named humans.
5. Plan the server certificates: one SBD Server Authentication certificate (course 1.6) each for SQL01, APPS01 (for its application API), and WEB01, with the host's DNS name in the subject alternative name. Each host requests its own in its build lesson through the course 1.6 process, so the private key never leaves it — the template allows exportable keys only where course 1.19's recovery plan says so. Course 1.6's autoenrollment GPO, linked at the domain root, already reaches every new member, so no other enrollment policy is created.

Check your result

```
# From AdminJumpBox01 (Tier 0): the course 1.11 KDS root key, the existing gMSAs, and the
role groups
```

```
Invoke-Command SBD-DC01 { Get-KdsRootKey | Select-Object KeyId, EffectiveTime }   # one
key, from course 1.11
Get-ADServiceAccount -Filter * | Select-Object Name, Enabled   # course 1.11's three; this
course's arrive with their hosts
Get-ADGroup -SearchBase 'OU=Groups,OU=SBD,DC=sbd,DC=test' -Filter * | Select-Object Name
```

WHAT SUCCESS LOOKS LIKE The register carries every VLAN 40 service row with its ID and owner — the SQL path restricted to APPS01 alone (F22), the WEB01-to-SQL01 deny (F14), and WEB01's repository, syslog and time rows — the course 1.11 KDS root key is the forest's only one, the role groups exist, and each server certificate is planned for the host that will request it.

Your record

Evidence to keep: Keep the service-path rows, the identity and group design, the KDS root key check, and the certificate plan.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Who may reach SQL01 on 1433?

REMEMBER Every path is a row; the database hears only SBD-APPS01. Service identities are managed accounts, not passwords. Access belongs to roles; each certificate is planned for its own host.

Microsoft · Group managed service accounts

NIST · SP 800-53 Rev. 5 security controls

Microsoft · Delegated Managed Service Accounts overview

Microsoft · Create a Key Distribution Service (KDS) root key

LESSON 02 / ABOUT 30 MIN

Build SBD-FS01 with least-privilege shares

> **WHY THIS MATTERS** The file server is where access control meets daily life. Build the shares against the role groups, require SMB signing on the server instead of trusting client defaults, and make 'who can read this' a one-lookup question.

In this lesson you will

- Build SBD-FS01 in its place and create the shares with role-group permissions and no individual grants.
- Confirm the SMB signing and encryption posture end to end.
- Verify access and denial per role from the fleet.

Follow the build

1. Build SBD-FS01 as a Windows Server virtual machine at 10.0.40.10/24, with the core's VLAN 40 interface as gateway (10.0.40.1 in this sample), both domain controllers for DNS, a separate data volume for the shares, and the hypervisor's time synchronization turned off. Join it straight into the Servers OU, so course 1.5's STIG policies, course 1.8's tier policies, course 1.10's event-forwarding GPO and course 1.14's Ring 3 update policy apply from its first boot, and check that it takes its time from a domain controller. Lessons 3 and 4 build SBD-SQL01 and SBD-APPS01 the same way.
2. Create the planned shares, granting NTFS permissions to the lesson-1 role groups only — share permissions stay simple, the filesystem carries the detail, and no named human appears in an access control list (ACL).
3. Enable access-based enumeration so users see only what their roles permit — the quiet UX of least privilege.
4. Confirm the SMB posture. The newest Windows client release (Pro, Enterprise, and Education) requires SMB signing for both inbound and outbound connections, but the newest Windows Server release requires it only for its own outbound connections — so require signing on SBD-FS01 as well (RequireSecuritySignature in its SMB server configuration), and opt the sensitive share into SMB encryption. Verify both on a live connection from a fleet machine rather than assuming the defaults.
5. Test as members of two different roles from fleet machines: each sees and opens exactly its shares, and the other's are invisible and, opened by path, denied. Turn on Audit Detailed File Share (failure) for SBD-FS01 in a GPO security-filtered to it, recorded as a documented addition as course 1.11 did for SBD-AUDIT01, so each refusal is event 5145 in its Security log; lesson 5 sends that event to the collector.

Check your result

```
# In a Tier 1 session on SBD-FS01 from SBD-ADMIN-SRV01
([adsisearcher]"(&(objectCategory=computer)(name=$env:COMPUTERNAME))").FindOne().Path
# ...OU=Servers,OU=SBD,DC=sbd,DC=test
w32tm /query /source   # a domain controller, never the local clock
Get-SmbShare | Select-Object Name, Path, FolderEnumerationMode, EncryptData
Get-SmbServerConfiguration | Select-Object RequireSecuritySignature   # expected True
icacls D:\Shares\Projects | Select-String 'SBD'
auditpol /get /subcategory:"Detailed File Share"   # Failure
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=5145} -MaxEvents 5 | Select-Object
TimeCreated, Message
# On the fleet machine, during the test session
```

```
Get-SmbConnection | Where-Object ServerName -like 'SBD-FS01*' | Select-Object ShareName,
Dialect, Signed, Encrypted
```

> **WHAT SUCCESS LOOKS LIKE** SBD-FS01 sits in the Servers OU with time from a domain controller, shares grant to role groups only, FS01 requires signing, a live connection from the fleet shows Signed True — or, on the encrypted share, Encrypted True, where Signed reads False because encryption, which also protects integrity, takes the place of signing — and the two-role test proves both the access and the invisible-denied behavior, with each refusal recorded as event 5145.

Your record

Evidence to keep: Keep the OU placement and time source, the share and NTFS permission reports, FS01's signing setting and the live connection's signing/encryption output, and the two-role test record with its 5145 events.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Where does the fine-grained permission detail live?

> **REMEMBER** Roles own the ACLs; humans own role memberships. Signing and encryption are verified on a live connection. Both the access and the denial are on record.

Microsoft · SMB security enhancements

Microsoft · SMB signing

Microsoft · Get-SmbConnection

Microsoft · MSFT_SmbConnection class (Signed and Encrypted)

Microsoft · What's new in Windows Server (newest release)

Microsoft · Advanced audit policy configuration (File System, Detailed File Share)

LESSON 03 / ABOUT 30 MIN

Build SBD-SQL01 behind its applications

WHY THIS MATTERS The database runs as a managed identity, encrypts its connections with a real certificate, and answers exactly one caller, SBD-APPS01 — the narrowest audience on the network, by design.

In this lesson you will

- Build SBD-SQL01 and run the database service under its gMSA.
- Bind the SBD Server Authentication certificate and force encrypted connections.
- Prove the one-caller rule: SBD-APPS01 connects; a VLAN 40 neighbour and the fleet are refused.

Follow the build

1. Build SBD-SQL01 (10.0.40.20) and the host of its one caller, SBD-APPS01 (10.0.40.30), as SBD-FS01 was built in lesson 2. Then, from AdminJumpBox01, create one gMSA per service, each allowed to one computer as course 1.11 did: SBD-SQL$ for SBD-SQL01$ and SBD-Apps$ for SBD-APPS01$ (sample names). Install the database engine with SBD\SBD-SQL$ as its service account and no password — the install itself demonstrates the managed account working where a password account used to live.
2. Add SBD-SQL01 to the SBD TLS Servers group, then on SBD-SQL01 request its SBD Server Authentication certificate through the course 1.6 process, with SBD-SQL01.sbd.test in the subject alternative name. Give the gMSA read on the certificate's private key — without it the service won't start with the certificate — select the certificate in SQL Server Configuration Manager, set Force Encryption to Yes, and restart the service; the chain validates against pki.sbd.test like every other certificate on this network.
3. Create the application databases with role-based database access for one login only, SBD\SBD-Apps$, the identity SBD-APPS01's services run as in lesson 4 — least privilege continues inside the engine, not just at its port. Record the databases and the logins they depend on: course 1.19 backs them up, so this course sets up no backup job.
4. Prove the audience: allow TCP 1433 in SBD-SQL01's host firewall from 10.0.40.30 alone (F22), and turn on its dropped-packet logging if it is off. A connection from APPS01 succeeds. The same attempt from SBD-FS01 is refused by SBD-SQL01's host firewall — the two share VLAN 40, so the router never sees it, which is exactly how SBD-WEB01 will be refused in lesson 4 — and one from a fleet machine dies at the router with a deny-log entry.

Check your result

```
# From AdminJumpBox01 (Tier 0): one gMSA per service, each allowed to one computer
New-ADServiceAccount SBD-SQL -DNSHostName SBD-SQL.sbd.test -
PrincipalsAllowedToRetrieveManagedPassword 'SBD-SQL01$'
New-ADServiceAccount SBD-Apps -DNSHostName SBD-Apps.sbd.test -
PrincipalsAllowedToRetrieveManagedPassword 'SBD-APPS01$'
# In a Tier 1 session on SBD-SQL01 from SBD-ADMIN-SRV01: identity, encryption,
certificate, firewall scope
Get-CimInstance Win32_Service -Filter "Name='MSSQLSERVER'" | Select-Object Name,
StartName, State   # StartName SBD\SBD-SQL$
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Microsoft SQL Server\*\MSSQLServer\
SuperSocketNetLib' | Select-Object ForceEncryption, Certificate   # 1 and the thumbprint
$cer = Get-ChildItem Cert:\LocalMachine\My | Where-Object Subject -like '*SBD-SQL01*' |
Export-Certificate -FilePath "$env:TEMP\sql01.cer"
certutil -verify -urlfetch $cer.FullName   # chain and revocation from pki.sbd.test
```

```
Get-NetFirewallPortFilter | Where-Object LocalPort -eq 1433 | Get-NetFirewallRule | Where-
Object { $_.Enabled -eq 'True' -and $_.Direction -eq 'Inbound' } | Get-
NetFirewallAddressFilter | Select-Object RemoteAddress   # 10.0.40.30 only
# From SBD-APPS01: expected TcpTestSucceeded True. From SBD-FS01 and from a fleet machine:
expected False.
Test-NetConnection SBD-SQL01.sbd.test -Port 1433
```

> **WHAT SUCCESS LOOKS LIKE** The service runs as the gMSA, Force Encryption is on with the issued certificate bound and its chain validating, the database role admits SBD\SBD-Apps$ alone, TCP 1433 is open to 10.0.40.30 only, APPS01 reaches it, and both other attempts fail with a record: SBD-FS01's at SQL01's host firewall, the fleet's at the router.

Your record

Evidence to keep: Keep the service identity proof, the encryption setting, certificate binding and urlfetch output, the database role grants, the firewall scope, and APPS01's successful connection paired with both refusals and their log entries.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What demonstrates the gMSA actually working?

> **REMEMBER** A managed identity runs the engine; no password exists to leak. Connections encrypt under the lab's own validated chain. One caller in: SBD-APPS01. Its VLAN 40 neighbours are refused at SQL01's host firewall and the fleet at the router.

Microsoft · Securing SQL Server

Microsoft · Group managed service accounts

Microsoft · Encrypt connections to SQL Server by importing a certificate

Microsoft · Configure Windows Firewall rules with Group Policy

LESSON 04 / ABOUT 40 MIN

Build SBD-APPS01 and SBD-WEB01

WHY THIS MATTERS The application server and the internal web server complete the dependency chain: fleet to web, web to application, application to database — each hop a register row, the web tier hardened against its Defense Information Systems Agency (DISA) guidance, and HTTPS served from the lab's own trust.

In this lesson you will

- Stand up SBD-APPS01's licensing service and application API on their registered paths.
- Build the domain-joined Linux Apache server the way courses 1.9 and 1.12 built theirs, hardened per its DISA Security Technical Implementation Guide (STIG) and serving HTTPS with the internal certificate.
- Prove the full chain: fleet → web → application → database, with the web-to-database shortcut refused.

Follow the build

1. On SBD-APPS01, built in lesson 3, and added to the SBD TLS Servers group, request its SBD Server Authentication certificate through the course 1.6 process and run both of its services as SBD\SBD-Apps$, or record an exception where the product can't use a gMSA: the licensing service, verified from the fleet on its registered vendor port (F12), and the application API on HTTPS 443 (F21), with its host firewall admitting the API from SBD-WEB01 (10.0.40.40) alone. APPS01 is the network's one SQL consumer — and the template for any future service: identity, row, evidence.
2. Build SBD-WEB01 at 10.0.40.40/24 from hashed install media of the Red Hat Enterprise Linux–family release course 1.9 chose, with both domain controllers for DNS and the hypervisor's time synchronization turned off. Before anything else, install course 1.3's chrony file (both domain controllers, maxdistance 16) and confirm a controller is selected with a small, steady offset; then apply course 1.9's client package, checked against its hashes, so SBD-REPO01 is its only package source (F19), and update from the current snapshot. Place the course 1.6 root in its system trust store by hand (/etc/pki/ca-trust/source/anchors, then update-ca-trust), its SHA-256 checked against course 1.6's record, as course 1.9 did for SBD-REPO01.
3. Join sbd.test with realm join into the Servers OU, as SBD-SPLUNK01 was joined in course 1.12, permit sign-in only to SBD Tier 1 Admins with realm permit -g, and give that group sudo through a drop-in file. Make the host firewall the register's copy: HTTPS 443 from the fleet (F13) and SSH from 10.0.60.20 alone. Then send its syslog over TLS to SBD-SYSLOG01 (F07) as course 1.10 enrolled SBD-REPO01: rsyslog-gnutls from the snapshot, the collector's certificate name checked, and SBD-WEB01 added to the collector's TCP 6514 sender list.
4. Request SBD-WEB01's SBD Server Authentication certificate through the course 1.6 process, with the key and request made on the host with openssl and SBD-WEB01.sbd.test in both subject and subject alternative name, and the key readable by root alone. Serve the internal application over HTTPS with its chain, and harden Apache against the DISA Apache Server 2.4 UNIX STIGs that course 1.9 staged — the 2,048-bit key floor from course 1.6's templates applies here as everywhere. Export the hardened configuration and hash it as the change record (course 1.19 backs up SBD-APPS01's and SBD-WEB01's configuration and certificates); lesson 5 measures the host.
5. Prove the chain from a fleet machine: the HTTPS page loads with a validating certificate, and its data travels fleet → WEB01 → APPS01 → SQL01, every hop a register row — on SBD-SQL01, sys.dm_exec_connections shows APPS01's session (10.0.40.30) with encrypt_option TRUE. Then

prove the shortcuts stay shut: a direct attempt from SBD-WEB01 to SQL01 on 1433 is refused by SQL01's host firewall, and the fleet still cannot touch 1433 at all — three courses of design visible in one browser request.

Check your result

```
# On SBD-WEB01 (Tier 1 SSH session from SBD-ADMIN-SRV01): time, packages, domain,
firewall, logging, web
chronyc sources -v; chronyc tracking   # ^* one controller, ^+ or ^- the other
dnf repolist --all   # expected: only the sbd-repo01 repositories enabled
realm list   # sbd.test, kerberos-member, permitted group SBD Tier 1 Admins
sudo firewall-cmd --get-active-zones
openssl s_client -connect SBD-SYSLOG01.sbd.test:6514 -verify_hostname SBD-
SYSLOG01.sbd.test </dev/null 2>/dev/null | grep 'Verify return code'   # 0 (ok): the SBD
root is trusted
logger -t sbd-web-proof 'SBD-WEB01 syslog over TLS test'
sudo apachectl -S   # one HTTPS site on 10.0.40.40:443
curl -sS -o /dev/null -w '%{http_code}\n' https://SBD-APPS01.sbd.test/   # use your
application's health path
timeout 5 bash -c '</dev/tcp/SBD-SQL01.sbd.test/1433' && echo open || echo 'refused or
timed out'   # expected: refused or timed out
# On SBD-ADMIN-SRV01: the issued certificate's chain and revocation
certutil -verify -urlfetch C:\Evidence\business-services\web01.cer
# From a fleet machine: the page answers, the database does not
Test-NetConnection SBD-WEB01.sbd.test -Port 443
Test-NetConnection SBD-SQL01.sbd.test -Port 1433   # expected: False
# On SBD-SQL01, in a query: APPS01's live session is encrypted
# SELECT client_net_address, encrypt_option, auth_scheme FROM sys.dm_exec_connections
WHERE client_net_address = '10.0.40.30'
```

> **WHAT SUCCESS LOOKS LIKE** The licensing path and the application API work as registered under the managed identity; SBD-WEB01 takes its time from the controllers and its packages only from SBD-REPO01, trusts the SBD root, is domain-joined with SSSD in the Servers OU, and sends its syslog to the collector over TLS; the Apache host serves HTTPS with a validating internal chain from a hardened, hashed configuration; and the end-to-end request proves fleet → web → application → database, encrypted to the database, with both shortcuts denied.

Your record

Evidence to keep: Keep APPS01's identity and path proof, WEB01's time, repository, trust, domain-join and syslog checks, its hardened config export with hashes, the certificate validation output, the end-to-end chain test with SQL01's connection record, and the refused WEB01-to-SQL01 attempt.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	

RECORD	YOUR OBSERVATION
Open items or findings	

Think it through

Where do WEB01's packages come from?

> **REMEMBER** APPS01 serves licensing and the application API; WEB01 joins the domain like the other Linux servers. The web tier is built the Linux way — chrony, SBD-REPO01, the SBD root by hand — hardened to its guidance and serving the lab's own trust. One page load proves fleet → web → application → database; the shortcuts stay shut.

Microsoft · Securing SQL Server

Microsoft · Group managed service accounts

NIST · SP 800-53 Rev. 5 security controls

Red Hat · Integrating RHEL systems directly with Windows Active Directory

chrony · chrony.conf(5)

Red Hat · Using shared system certificates

firewalld · documentation

Apache HTTP Server 2.4 · Security tips

LESSON 05 / ABOUT 30 MIN

Reconcile, scan, and hand off

> **WHY THIS MATTERS** The business services close the way every build course has: the register reconciled against live behavior, the new systems baselined, the inventory updated — and for the first time, the network is functionally complete.

In this lesson you will

- Enrol the new servers and reconcile the full register against observed permits and denies.
- Baseline the four service hosts with the tool each platform takes.
- Update the living records and write the course's handoff.

Follow the build

1. Enrol the three Windows servers in the record: from AdminJumpBox01, add SBD-FS01, SBD-SQL01 and SBD-APPS01 to SBD Event Sources (course 1.10). On the collector, in a Tier 1 session, widen the WinRM 5985 scope to 10.0.40.10, 10.0.40.20 and 10.0.40.30 (F17) and add a query to SBD-ForwardedSecurity for SBD-FS01's event 5145, filtered to that computer as course 1.11 did for SBD-AUDIT01. Restart the three servers and confirm their events reach ForwardedEvents; SBD-WEB01 already sends its syslog over TLS (lesson 4).
2. Run the full-register reconciliation one more time: every business-service row proven from its real source, every adjacent deny probed and logged, including WEB01 to SQL01 — the VLAN 40 rows join the register's evidence beside the earlier courses' proofs.
3. Take the first baselines. SBD-FS01, SBD-SQL01 and SBD-APPS01 get course 1.4's SCC and Windows Server benchmark versions, run locally in Tier 1 sessions from SBD-ADMIN-SRV01 as course 1.14 did for SBD-WSUS01, plus DISA's SQL Server STIGs for the instance, staged across the air gap as course 1.4 taught and answered in a STIG Viewer checklist. SBD-WEB01 gets SCC for Linux and the distribution benchmark from course 1.9's set in C:\Staging\DISA\linux, scanned the course 1.9 way, with the Apache STIG items in its checklist. Classify every open item as course 1.4 taught — fix by GPO, deviation candidate or POA&M candidate, with fix now (no GPO applies to Linux) for SBD-WEB01. Credentialed vulnerability scanning arrives with course 1.17, coming soon.
4. Update the living records: the four hosts' planned rows in the SBD-MGMT01 inventory turn active through its change log (course 1.15), with their identities, certificates, rings and baselines; the new rows' owners go into the register; and any service-hardening deviations go to the owner of the exception record, C:\Evidence\exception-record.csv on AdminJumpBox01.
5. Write the handoff: every modeled system that isn't marked coming soon is now built, watched, patched, and evidenced. This course sets up no backups: course 1.19 defines and schedules them for the VLAN 40 servers — SBD-FS01's share data with its NTFS permissions, SBD-SQL01's databases and the logins they depend on, and SBD-APPS01's and SBD-WEB01's configuration and certificates — and proves the network can come back from the dead; then path 2 takes the whole record to authorization.

Check your result

```
# On SBD-SYSLOG01 (Tier 1 session): the new sources report
wecutil gr SBD-ForwardedSecurity   # SBD-FS01, SBD-SQL01 and SBD-APPS01: Active
Get-WinEvent -FilterHashtable @{LogName='ForwardedEvents'; StartTime=(Get-Date).AddHours(-1)} | Group-Object MachineName | Select-Object Name, Count
```

```
Select-String -Path D:\Logs\Syslog\*\*.log -Pattern 'sbd-web-proof' | Select-Object -First
1
# On SBD-ADMIN-SRV01: collect the course's records and hash them
Get-ChildItem C:\Evidence\business-services -Recurse -File | Select-Object FullName,
LastWriteTime
Get-ChildItem C:\Evidence\business-services -Recurse -File | Get-FileHash -Algorithm
SHA256 | Export-Csv C:\Evidence\business-services-hashes.csv -NoTypeInformation
```

> **WHAT SUCCESS LOOKS LIKE** The three Windows servers forward their events and SBD-WEB01 its syslog, the register covers every VLAN 40 row with paired permit/deny proof, all four hosts carry first baselines with classified findings, and inventory, register, and exception record all reflect the finished services.

Your record

Evidence to keep: Finish the checklist: the enrolment proof, the reconciliation record, the baselines and checklists with classified findings, the updated living records, and the handoff note to course 1.19.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why re-run the whole register reconciliation now?

> **REMEMBER** Register and reality reconcile across the finished network. Every service host carries its platform-appropriate evidence. One course remains: proving the network can be brought back.

NIST · SP 800-53 Rev. 5 security controls

DISA · Security Content Automation Protocol (SCC and SCAP benchmarks)

DISA · SRG/STIG tools (SCC and STIG Viewer)

Microsoft · Get-FileHash

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the service paths and identities	☐ yes	
02 Build SBD-FS01 with least-privilege shares	☐ yes	
03 Build SBD-SQL01 behind its applications	☐ yes	
04 Build SBD-APPS01 and SBD-WEB01	☐ yes	
05 Reconcile, scan, and hand off	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.18. Course 1.19 closes the Build path by backing up and proving recovery — directory, CA, policy, logs, and the VLAN 40 services built here — before authorization begins.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.